

Certifiable Support Budgets versus Intrinsic Support in Quantum Compilation

ORION-01 Paper B — bounded manuscript V3

Status: candidate successor to the frozen V2 text; independently readable; no external review or submission authority claimed

Abstract

A normal-form theorem can prove that an exact optimum exists below a support ceiling without proving that the ceiling is necessary for the compiler. We formalize that distinction for a fixed objective and a named proof system.

For an instance family F and objective C , let $\kappa(F, C)$ be the smallest integer k such that every admitted instance has a C -optimal solution with support at most k . This is a mathematical property of the instance family, not an epistemic claim about current evidence. For a proof system P , let $\beta_P(F, C)$ be the smallest support budget that P can certify uniformly. A sound proof system gives $\kappa(F, C) \leq \beta_P(F, C)$; equality requires a separate lower-bound witness for the compiler, while strict inequality is a separation between the proof language and the compiler.

We instantiate P as the rank-only zero-sum deletion system: signatures lie in a fixed elementary binary group, a nonempty zero-signature subsequence may be deleted, feasibility and objective dominance are declared premises, and the support conclusion uses only binary rank. For the same frozen weighted support objective, R6M has intrinsic support two and rank-only budget two. R6I has intrinsic support one while its rank-only deletion system has budget five, giving a strict proof-language separation. A product statement is included only as a definitional amplification under an explicit no-cross-component composition rule. The paper does not claim general certificate complexity, production move completeness, physical resource advantage, or a new zero-sum invariant.

1. Why the distinction matters

A certificate can be exact, sound, and useful while still being incomplete. Confusing the certificate's numerical ceiling with the compiler's intrinsic minimum can turn a limitation of a proof language into a false lower bound. ORION-01 therefore separates three objects:

1. the compiler and its exact objective;
2. the mathematical intrinsic support of exact optima;
3. the support budget certified by a named proof system.

The term **certifiable support budget** is used deliberately. It is not the computational-complexity notion called certificate complexity.

2. Fixed optimization semantics

Let F be a family of finite exact compilation instances. Let $X(I)$ be the feasible set for instance I , let $C_I: X(I) \rightarrow \mathbb{R}$ be the fixed objective, and let $\sigma_I(x)$ be the support statistic under study.

Definition 1 (intrinsic optimal support)

For an instance I , define

$$\kappa(I; C) = \min \{ \sigma_I(x) : x \in \operatorname{argmin}_{\{y \in X(I)\}} C_I(y) \}.$$

For a family F , define the uniform intrinsic support

$$\kappa(F; C) = \sup_{\{I \in F\}} \kappa(I; C)$$

when the supremum is finite.

This definition is mathematical. A witness, theorem, or exact search may establish its value, but the value is not defined by what has been proved so far.

Definition 2 (proof-system certifiable support budget)

A proof system P is a fixed collection of admissible premises, inference rules, and conclusion forms. Define

$\text{beta}_P(F;C) = \min \{ k : P \text{ proves, uniformly for every } I \text{ in } F, \text{ that some } C\text{-optimal } x \text{ has } \sigma_I(x) \leq k \}.$

If no finite uniform bound is derivable, set $\text{beta}_P(F;C)=\text{infinity}$.

Proposition 1 (soundness inequality)

If P is sound for (F,C) , then

$\kappa(F;C) \leq \text{beta}_P(F;C).$

Proof

A sound derivation of budget k establishes, for every admitted instance, a C -optimal solution with support at most k . The minimum support among C -optimal solutions is therefore at most k for every instance. Taking the family supremum and then the smallest provable k gives the result. QED.

The reverse inequality does not follow from soundness. A proof language may be too coarse to derive the compiler's best support.

3. The rank-only zero-sum deletion system

Fix an elementary binary group $H=F_2^d$ and an instance-level admissible alphabet $A \subseteq H$. A sequence means an ordered list over A . A subsequence may select arbitrary positions; it need not be contiguous. The empty sequence is admitted, and a zero-sum-free sequence is one with no nonempty zero-sum subsequence.

The proof system P_{rank} contains the following rule.

Rank deletion rule

For a supported object with signature sequence W :

1. the total signature of W is nonzero;
2. deleting coordinates whose signatures sum to zero preserves feasibility of the whole instance;
3. the deletion does not increase the fixed objective C ;
4. the conclusion may use only the ambient binary rank d , not a finer compiler-specific search or a smaller restricted alphabet invariant.

If $|W| > d$, binary linear dependence supplies a nonempty zero-sum subsequence. Because the total is nonzero, the subsequence is proper. The declared deletion is therefore legal and nonincreasing.

Theorem 2 (rank-only budget)

For a family whose declared rank rule has dimension d and whose premises hold uniformly,

$\text{beta}_{P_{\text{rank}}}(F;C) \leq d.$

If the proof-system model admits a zero-sum-free basis sequence and no additional rule can reduce it, then $\text{beta}_{P_{\text{rank}}}(F;C)=d$.

This is a theorem about the named proof system. The basis obstruction is not automatically an intrinsic compiler lower bound: the compiler may have an optimum whose signature sequence never realizes that obstruction.

4. Equality and separation

Definition 3 (certificate sharpness)

P is support-sharp for (F,C) when

$$\text{kappa}(F;C) = \text{beta_P}(F;C).$$

Definition 4 (proof-language separation)

There is a strict support separation when

$$\text{kappa}(F;C) < \text{beta_P}(F;C).$$

A strict separation says that the proof language certifies a weaker support budget than the compiler actually needs. It does not say the compiler is computationally easy, and it does not compare runtime complexity classes.

5. Same-objective R6M control

The R6M and R6I comparisons use the same frozen weighted support objective C; neither side changes the objective to obtain a favorable support value.

For the one-Tag, three-block R6M family, the rank-only signature group has dimension two. Paper A's compiler-specific semantic and objective premises establish

$$\text{beta_P_rank}(R6M;C)=2.$$

The intrinsic upper/lower result is bound directly to the following parent artifacts:

- upper implementation: `research/extensions/orion-qg/paper_a_a1_multitag_tare.py`;
- upper result: `research/extensions/orion-qg/PAPER_A_A1_MULTITAG_TARE_RESULTS_2026-08-24.json`;
- lower parent: `research/extensions/orion-qg/QG18_TARE_KAPPA_RESULTS.json`;
- all-size upper parent: `research/extensions/orion-q/MAX_R6S_ALL_N_COMPOSITION_RESULTS.json`.

The upper theorem gives an optimum with support at most two for every admitted size, while the exact lower witness excludes support one for the declared witness instance under the same objective. Therefore

$$\text{kappa}(R6M;C)=2=\text{beta_P_rank}(R6M;C).$$

This is the bounded sharp control. It is not a general sharpness theorem for all MultiTag grammars.

6. R6I strict separation

For the frozen R6I family, the named rank-only deletion language exposes a five-dimensional binary signature space and admits a zero-sum-free basis obstruction within that proof-system abstraction. Hence

$$\text{beta_P_rank}(R6I;C)=5.$$

The compiler-side exact witness and upper construction establish, under the same objective,

$$\text{kappa}(R6I;C)=1.$$

Consequently

$$\text{kappa}(R6I;C)=1 < 5=\text{beta_P_rank}(R6I;C).$$

The gap is a proof-language separation: binary rank alone loses compiler structure that an exact compiler argument can exploit. The statement does not imply that every conceivable proof system needs budget five.

The parent evidence for the intrinsic value and the proof-system basis obstruction must remain content-hash bound in the release manifest. The implementation-independent checker in this package checks the abstract rank/deletion logic but does not regenerate those production witnesses.

7. Definitional product amplification

Let $F1$ and $F2$ be independently composed instance families with product objective

$$C((x1, x2)) = C1(x1) + C2(x2)$$

and additive support

$$\text{sigma}((x1, x2)) = \text{sigma1}(x1) + \text{sigma2}(x2).$$

Assume the composition exposes no cross-component compiler move and the proof system has no cross-component inference rule. Then optimality and certification factor componentwise.

Proposition 3 (componentwise product identity)

Under those assumptions,

$$\text{kappa}(F1 \times F2; C) = \text{kappa}(F1; C1) + \text{kappa}(F2; C2)$$

and

$$\text{beta}_{\{P1 \times P2\}}(F1 \times F2; C) = \text{beta}_{\{P1\}}(F1; C1) + \text{beta}_{\{P2\}}(F2; C2).$$

Proof

The additive objective and absence of cross-component moves make a product optimum exactly a pair of component optima. Minimum optimal support therefore adds. The product proof system can derive exactly the component budgets and no cross-component reduction, so its minimum uniform budget also adds. QED.

For m independent copies of R6I, this gives intrinsic support m and rank-only budget $5m$. The absolute gap is $4m$ and the ratio is five. This is a definitional amplification of the one-component separation, not an independent empirical or mathematical discovery.

8. Distinct asymptotic readings

Two asymptotic statements must not be conflated.

1. **Fixed family parameter, growing instance size.** For fixed Tag/rank parameter, an all-size theorem may give a support budget independent of circuit size.
2. **Growing family parameter.** If the signature rank or number of independent components grows, the rank-only budget can grow as well.

A bounded-in-size theorem for fixed rank does not imply a uniform bound when the rank itself tends to infinity.

9. Relation to donor work

The following are donor-owned context rather than residual contributions:

- Davenport and restricted/subset zero-sum constants;
- binary linear dependence;
- general sparse optimal-solution theorems;
- Pauli symplectic representations;
- generic distinctions between a phenomenon and the strength of a proof system.

The bounded residual is the explicit same-objective pair of compiler controls: equality for R6M and strict rank-only separation for R6I, together with the semantic boundary that prevents the certificate budget from being misreported as intrinsic support.

The paper does not claim a general theory of proof complexity, a lower bound against all proof systems, a new certificate-complexity measure, or a production move-completeness result.

10. Reproducibility

The release package must bind:

1. the exact objective and instance-family definitions;
2. the parent upper/lower artifacts for `kappa(R6M;C)=2`;
3. the parent artifacts for `kappa(R6I;C)=1`;
4. the rank-only proof-system definition and basis obstructions;
5. the implementation-independent theorem checker;
6. the active adverse terminal `CANNOT_CHECK_MOVE_COMPLETENESS` from PR #1602.

`proof_checker_v3.py` imports no production compiler code. It checks small finite models of the definitions, the soundness inequality, binary and cyclic boundary cases, exact Restore sensitivity, and componentwise arithmetic. Such replay catches statement/implementation mismatches in the abstract layer; it is not external validation of parent compiler evidence.

11. Limitations

1. `beta_P` is relative to an explicitly fixed proof language.
2. A smaller alphabet-aware invariant may improve on ambient rank without reaching intrinsic support.
3. The exact R6M and R6I values inherit the scope and assumptions of their parent evidence.
4. The product identity requires additive objectives and an explicit no-cross-component rule.
5. Fixed-rank and growing-rank asymptotics are different regimes.
6. Support is not gate count, depth, runtime, qubits, noise performance, or physical advantage.
7. No claim is made outside the frozen objective and instance families.
8. No larger search under the old Round-3 identity is authorized.
9. Author-side closeout is not external peer review or novelty authority.

12. Conclusion

Intrinsic optimal support and a proof system's certifiable support budget answer different questions. Soundness gives `kappa<=beta`; equality needs a compiler lower witness, while strict inequality diagnoses proof-language incompleteness. R6M is the bounded equality control and R6I is the bounded strict separation for the same objective. The result is deliberately narrow and does not promote the old capped search or the underlying zero-sum invariant.

Bounded disposition

`BOUNDED_PAPER_RETAINED`

The production-completeness successor is a separate, prospectively frozen exact-compute programme.